

CSCD602:

Advanced Software Engineering

Session 3 – Technical Debt

By

Solomon Mensah (PhD)

Dept. of Computer Science

smensah03@ug.edu.gh



UNIVERSITY OF GHANA

Session Outline

The key topics to be covered in the session are as follows:

- Understanding Technical Debt
- Causes of Technical Debt
- Types of Technical Debt
- Technical Debt Lifecycle
- Impact of Technical Debt on Software Systems
- Measuring and Identifying Technical Debt
- Strategies for Managing and Reducing Technical Debt
- Best Practices for Preventing Technical Debt

Learning Outcomes

At the end of the session, you will be able to :

- Define and explain technical debt.
- Identify its causes and types.
- Understand the lifecycle and impact on software systems.
- Measure and detect technical debt.
- Apply strategies to manage and reduce it.
- Implement best practices to prevent technical debt.

Definition

- Technical debt:
 - a fundamental concept that describes the long-term cost incurred when software developers take shortcuts or implement suboptimal solutions during system development in order to achieve short-term goals such as faster delivery.
- Self-admitted technical debt (SATD):
 - a type of technical debt where developers intentionally document limitations, shortcuts, or temporary solutions in the code using comments that indicate the need for future correction or improvement.

Constraints Faced

In software projects, developers often face constraints such as:

- Tight deadlines
 - Limited resources
 - Changing requirements
 - Market pressure for rapid delivery
-
- Because of these constraints, developers may implement temporary solutions instead of optimal ones.
 - While this allows the software to be delivered quickly, it introduces future maintenance challenges that must eventually be addressed.

Why Technical Debt exists?

Technical debt is common in software development due to several practical factors:

- **Time-to-Market Pressure:** Companies often release products quickly, so developers may prioritize speed over code quality.
- **Incomplete Requirements:** Changing or unclear requirements can lead to poorly structured implementations.
- **Rapid Prototyping:** Quick prototypes are sometimes used in production without proper redesign.

Why Technical Debt exists?

- **Lack of Experience:** Inexperienced developers may produce inefficient or poorly organized code.
- **Poor Communication:** Miscommunication among developers, managers, and stakeholders can result in weak architectural decisions.
- **Technology Evolution:** As technologies change, older systems may become outdated and accumulate technical debt.

The Technical Debt Quadrant

The Technical Debt Quadrant, introduced by Martin Fowler, classifies technical debt based on whether decisions were intentional or accidental and whether they were careful or careless.

- **Reckless & Deliberate:**

- Developers knowingly use poor solutions without planning to fix them.

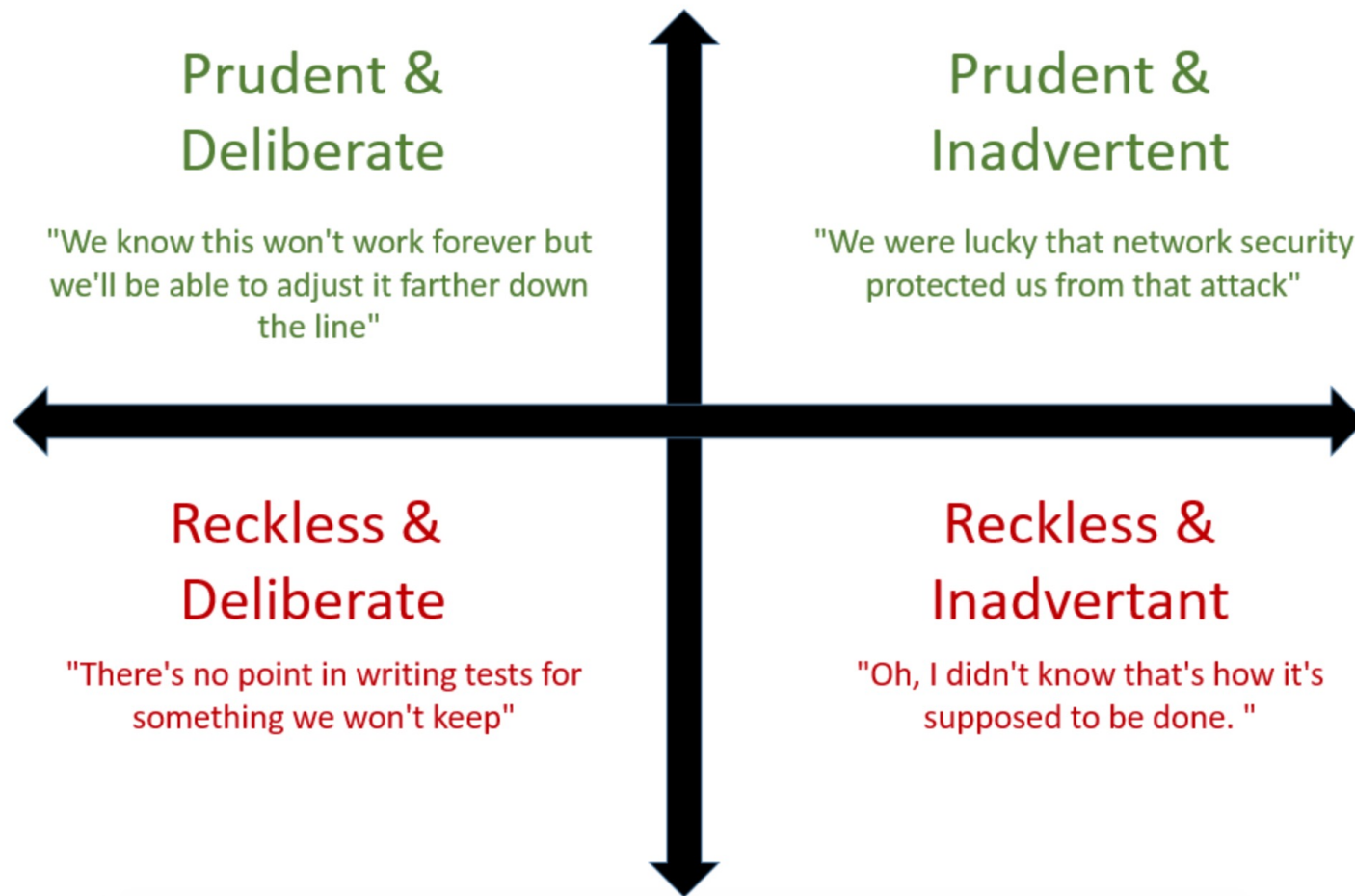
- **Prudent & Deliberate:**

- Developers take shortcuts intentionally but plan to improve the code later.

The Technical Debt Quadrant

- **Reckless & Inadvertent:**
 - Poor solutions are created unintentionally due to lack of knowledge.
- **Prudent & Inadvertent:**
 - Developers discover better solutions after the system has already been implemented.
- This model helps teams understand the causes of technical debt and how to manage it effectively.

The Technical Debt Quadrant



Causes of Technical Debt

Technical debt can arise from several situations:

- **Tight deadlines**
Developers rush to release software quickly.
- **Poor planning or design**
Lack of proper architecture leads to inefficient code.
- **Lack of documentation**
Future developers cannot easily understand the system.
- **Changing requirements**
Software must be modified frequently.
- **Lack of testing**
Undetected bugs remain in the system.
- **Inexperienced developers**
Developers may not follow best coding practices.

Effects of Technical Debt

If not managed properly, technical debt can cause:

- Slow development of new features
- Increased bugs and system failures
- Higher maintenance cost
- Poor system performance
- Reduced software quality

Characteristics of Technical Debt

Technical debt typically has the following characteristics:

1. Short-Term Benefit

- Developers gain immediate advantages such as faster delivery or meeting tight deadlines.

2. Long-Term Cost

- The system becomes harder to modify, extend, or maintain.

3. Accumulated Interest

- Over time, technical debt leads to:
 - Increased bug frequency
 - Higher maintenance costs
 - Reduced development speed

4. Hidden Complexity

- As technical debt accumulates, the software system becomes more complex and difficult to understand.

Categories of Technical Debt (Intentional vs Unintentional)

Technical debt can be classified based on intent and awareness.

Intentional Technical Debt

- This occurs when developers knowingly choose a quick solution with the intention of fixing it later.
- Referred to as the Self-admitted Technical Debt (SATD)
- Example:
 - Hardcoding temporary logic to release a feature quickly.

Unintentional Technical Debt

- This occurs when developers introduce poor solutions unknowingly due to lack of knowledge or poor design practices.
- Example:
 - Writing inefficient algorithms due to lack of experience

What is SATD?

- SATD refers to technical debt that developers explicitly acknowledge in the source code comments or documentation when they know that a part of the code is incomplete, poorly implemented, or requires future improvement.

Example of SATD

- Developers often mark SATD using comments such as:
 - // TODO: optimize this algorithm later
 - // FIXME: temporary solution, needs proper validation
 - // HACK: quick fix for production issue
- These comments indicate that the developer recognizes the presence of technical debt and plans to address it in the future.

Characteristics of SATD

- Explicitly documented by developers
- Appears in source code comments
- Indicates known issues or shortcuts
- Helps researchers and teams identify and manage technical debt

Types of Technical Debt in Software Systems

1. Code Debt

- Occurs when developers write poorly structured or messy code, often due to rushing or lack of experience.
- Examples: duplicate code, large complex functions, poor variable names.
- When it occurs: during fast development or when coding standards are ignored.

2. Architecture / Design Debt

- Occurs when the system architecture is poorly designed or not updated as the system grows.
- Examples: monolithic systems when modular design is needed, tightly coupled components.
- When it occurs: during early system design or when quick architectural decisions are made.

Types of Technical Debt in Software Systems

3. Test Debt

- Occurs when software is released with insufficient testing.
- Examples: lack of automated tests, low test coverage, missing integration tests.
- When it occurs: when teams skip testing to meet deadlines.

4. Documentation Debt

- Occurs when documentation is missing or outdated.
- Examples: no API documentation, outdated user manuals, missing system diagrams.
- When it occurs: when teams focus only on coding and ignore documentation.

Types of Technical Debt in Software Systems

5. Infrastructure Debt

- Occurs when outdated technologies or development environments are used.
- Examples: old frameworks, unsupported libraries, outdated servers.
- When it occurs: when systems are not regularly upgraded.

6. Process Debt

- Occurs when development practices are inefficient.
- Examples: no code reviews, poor version control, weak sprint planning.
- When it occurs: when teams lack proper development processes.

Lifecycle of Technical Debt

The correct lifecycle stages of Technical Debt, based on the descriptions provided, are:

- Introduction Stage: Debt appears due to shortcuts or poor design choices made to speed up delivery.
- Accumulation Stage: Debt grows as teams implement more quick fixes and workarounds over time.
- Interest Stage: The "cost" of the debt manifests as developers spend extra time on bug fixes and maintenance instead of new features.
- Crisis Stage: The system becomes extremely fragile and difficult to update, potentially reaching a point where maintenance costs outweigh benefits.

Measuring Technical Debt

Teams measure technical debt using software quality metrics such as:

- Code Complexity: e.g., Cyclomatic Complexity.
- Code Duplication: repeated sections of code.
- Test Coverage: percentage of code tested automatically.
- Code Smells: signs of deeper code design problems.

Common Tools:

- SonarQube
- CodeClimate
- ESLint
- PMD

Strategies for Managing and Reducing Technical Debt

Effective management of technical debt is essential for sustainable software development.

1. Refactoring is a disciplined process of restructuring existing computer code to improve its internal structure, readability, and maintainability without altering its external behavior or functionality.

Examples include:

- Simplifying complex methods
- Removing duplicate code
- Improving naming conventions.

Code Reviews

2. **Code reviews** are systematic, peer-driven evaluations of software code designed to improve quality, ensure standards compliance, and share knowledge. They act as a critical quality assurance step to identify bugs, security gaps, and logic errors before code is merged. Effective reviews involve constructive, respectful feedback focusing on clarity, test coverage, and functionality

Benefits include:

- Improved code quality
- Knowledge sharing
- Early detection of defects.

Automated testing

3. **Automated testing** is the use of specialized software tools to execute pre-scripted tests on an application, comparing actual outcomes with expected results without manual intervention. Implementing automated testing improves software reliability.

Types include:

- Unit testing
- Integration testing
- System testing.

Continuous Integration

4. **Continuous Integration (CI)** is a DevOps practice where developers frequently merge code changes into a central repository, triggering automated builds and tests. By automating validation, CI helps detect bugs early, reduces integration conflicts, and improves software quality, ensuring a faster, more reliable development.

Benefits include:

- Early error detection
- Reduced integration issues.

Improved Software Architecture

5. Software architecture defines the fundamental, high-level structure of a system, encompassing its components, their relationships, and the principles guiding its design and evolution. Using good architectural practices reduces long-term complexity.

Examples include:

- Microservices architecture
- Modular design
- Layered architecture

Technical Debt Backlog

6. A Technical Debt Backlog is a prioritized list of refactoring, code improvements, and structural changes needed to address compromises made to accelerate delivery. Teams should track technical debt as part of the **project backlog**.

Developers can schedule tasks such as:

- Refactoring
- Improving test coverage
- Updating documentation

Impact of Technical Debt on Software Projects

If technical debt is poorly managed, it can negatively impact projects by:

- Increasing maintenance costs
- Slowing down development
- Reducing software reliability
- Increasing system complexity
- Reducing developer productivity.

Importance of Managing Technical Debt

Proper management of technical debt helps organizations:

- Maintain high software quality
- Improve system reliability
- Reduce long-term maintenance costs
- Increase development productivity
- Enhance scalability of software systems.

Thank you

